

开发日报

日期: 2026年6月29日 (星期一)

今日定位: 内容组织日 (研发计划 4.2 — 阶段3第3天 / 共19天)

组长/汇报人: 孟之语

组员: 于贺、甘毅、王佳杭

1 今日总览

项目	内容
日期	2026年6月29日
阶段	阶段3 — 语义增强、特殊块保护、内容组织
今日定位	内容组织日: 实现关键词、主题标签、管理标签、摘要生成
里程碑	M3 — 核心智能体闭环初步可用
全组状态	正常, 全员在岗

2 晨会记录 (09:30-09:50)

2.1 昨日回顾 (6月28日 语义增强日)

孟之语汇报:

- 昨日完成sentence-transformers MiniLM模型的接入, 384维embedding, 模型大小118MB
- 语义边界评分算法已实现: $1 - \text{cosine_similarity}(\text{前段}, \text{后段})$, 阈值0.45
- 语义边界已集成到分段流程中, 作为弱边界信号

于贺汇报:

- 昨日协助完成了strategy_info元数据的结构和填充逻辑
- 每个chunk现在都标注了method (heading_boundary / semantic_boundary / length_split / merge) 和详细reason

甘毅汇报:

- 昨日完成了jieba TF-IDF关键词提取的初步实现
- 发现单纯的TF-IDF提取的关键词偏泛化 (如"文档"、"内容"等高频词)
- 需要今天加入停用词过滤和词性筛选

王佳杭汇报:

- 昨日协助孟之语完成了语义边界的阈值调试
- 使用3份样例文档测试了0.35/0.40/0.45/0.50四档阈值
- 阈值0.45在分段粒度和语义完整之间取得最佳平衡

2.2 今日任务分配

编号	任务	负责人	预计工时	优先级
T-7.1	完善内容标签体系设计（主题标签 + 管理标签）	甘毅	3h	P0
T-7.2	实现RuleBasedTagger完整版（关键词 + 主题标签）	甘毅	4h	P0
T-7.3	实现抽取式摘要生成（Lead-1 + TF-IDF句子打分）	甘毅	3h	P0
T-7.4	实现正则实体识别（百分比/日期/机构/指标/阈值）	甘毅	2h	P1
T-7.5	设计LLMClient接口，预留LLM增强模式	甘毅、孟之语	3h	P1
T-7.6	实现ContentOrganizer主流程编排	孟之语	3h	P0
T-7.8	编写关键词提取单元测试	于贺	2h	P1
T-7.9	评估数据集的问题标注（为阶段4做准备）	王佳杭	4h	P1
T-7.10	语义分段边界参数的网格搜索实验	王佳杭	3h	P2

2.3 晨会关键决策

决策1：标签体系分级

经讨论，确定三级标签体系：

- ```
1 第一级 - 内容标签 (What)
2 |— 主题标签 (topic_tags) : 回答"这段在讲什么"
3 | 示例：智能分段、RAG检索、标题感知
4 |— 关键词 (keywords) : 回答"核心术语是什么"
5 | 示例：chunk、embedding、Recall@k
6 |— 实体标签 (entity_labels) : 回答"涉及哪些实体"
7 | 示例：{类型：percentage, 值：85%}
8
9 第二级 - 管理标签 (Meta)
10 |— 来源标签 (source_type) : 文档、章节、表格
11 |— 状态标签 (status) : 草稿、已验证、待审核
12 |— 版本标签 (version) : v1.0、v2.0
13
14 第三级 - 质量标签 (Quality)
15 |— 置信度 (confidence) : high / medium / low
16 |— 异常标记 (anomaly) : oversized / fragmented / missing_title
17 |— 审核状态 (review) : passed / needs_review / rejected
```

### 决策2：规则模式 vs LLM模式的分工

- 规则模式（默认）：不依赖任何API Key，使用jieba + TF-IDF + 正则，开箱即用
- LLM模式（增强）：检测环境变量OPENAI\_API\_KEY，自动升级为DeepSeek生成
- 降级策略：LLM调用失败时自动回退到规则模式，不影响服务可用性

### 决策3：RuleBasedTagger的接口约定

与孟之语和于贺对齐后确定：

- `extract_tags(content) → list[str]` — 提取5-10个关键词/主题标签
- `generate_summary(content) → str` — 生成≤80字的抽取式摘要
- `extract_entities(content) → list[dict]` — 提取结构化实体

### 3 详细开发记录

#### 3.1 内容标签体系设计与实现（负责人：甘毅，3h）

##### 3.1.1 设计思路

标签体系的设计面临一个核心矛盾：标签既要足够具体（对检索有用），又不能过于泛化（如"文档"、"内容"这类万能标签）。解决方案是引入层次化过滤：

- 1 原始候选词（TF-IDF Top-20）
- 2     ↓ 停用词过滤（去掉"的"、"是"、"本文"等）
- 3     ↓ 词性筛选（保留名词、动名词、英文缩写）
- 4     ↓ 长度过滤（2-8个字符，过滤过短/过长）
- 5     ↓ TF-IDF权重排序（取Top-5~10）
- 6     ↓ 标签去重（基于编辑距离，合并相似标签）
- 7     → 最终标签列表

##### 3.1.2 停用词表设计

在jieba默认停用词表基础上，增加了学术/技术文档常见泛化词：

- 1 | \_STOP\_WORDS（约50个中英文通用停用词：虚词、代词、常见动词等）

##### 3.1.3 词性筛选规则

使用jieba.cut进行分词，过滤停用词和长度<2的词，保留中英文有效词。同时使用bigram组合策略，优先选取有意义的双词短语作为标签。

- 1
- 2 这一筛选显著提升了标签质量。例如"本项目旨在构建面向RAG的智能分段系统"这句话，原始TF-IDF Top-5是"项目"、"旨在"、"构建"、"面向"、"RAG"，经过词性筛选后保留"项目"、"RAG"（eng类别保留），过滤掉了动词性的"旨在"、"构建"、"面向"。
- 3
- 4 ##### 标签去重算法
- 5
- 6 使用编辑距离合并相似标签：
- 7
- 8 ```python
- 9 # 使用bigram短语组合策略去重和优选标签

#### 3.2 RuleBasedTagger 完整实现（负责人：甘毅，4h）

##### 3.2.1 类设计

- 1 class RuleBasedTagger:
- 2     """基于规则的内容标注器（零依赖、零API Key）
- 3
- 4     功能:
- 5     - extract\_tags: jieba分词 → TF-IDF权重 → 多层过滤 → 关键词标签
- 6     - generate\_summary: Lead-1首句 + TF-IDF句子排序 → 抽取式摘要
- 7     - extract\_entities: 正则匹配 → 结构化实体列表
- 8
- 9     设计原则:
- 10     1. 不依赖任何外部API，纯本地运行
- 11     2. 中文优先，兼容英文混合文本
- 12     3. 所有设计支持空输入，返回空等通用条件

```
13 """
14
15 def __init__(self, stop_words=None, max_tags=10):
16 self.stop_words = stop_words or DEFAULT_STOP_WORDS
17 self.max_tags = max_tags
18 self._tfidf = TfidfVectorizer(
19 tokenizer=self._tokenize,
20 max_features=100
21)
```

3.2.2 extract\_tags 实现细节

输入：任意长度的中文文本  
输出：5-10个关键词/主题标签列表

处理流程：jieba分词 -> 停用词过滤 -> TF-IDF权重计算(可选文档级IDF) -> bigram组合优选 -> 取Top-N标签

3.2.3 标签生成的边界条件测试

| 测试用例 | 输入                                            | 期望输出   | 实际输出                                               | 状态 |
|------|-----------------------------------------------|--------|----------------------------------------------------|----|
| 正常文本 | "面向RAG的智能分段系统，基于标题感知和语义边界进行文档分段"              | ≥5个标签  | ["RAG", "智能分段", "标题感知", "语义边界", "文档分段"]            | OK |
| 空文本  | ""                                            | 空列表    | []                                                 | OK |
| 纯英文  | "RAG segmentation using embedding similarity" | 英文标签   | ["RAG", "segmentation", "embedding", "similarity"] | OK |
| 中英混合 | "使用sentence-transformers进行embedding计算"        | 中英混合   | ["sentence-transformers", "embedding"]             | OK |
| 极短文本 | "智能分段"                                        | 1-2个标签 | ["智能分段"]                                           | OK |
| 重复内容 | "分段分段分段分段分段"                                  | 去重后1个  | ["分段"]                                             | OK |

3.3 抽取式摘要生成（负责人：甘毅，3h）

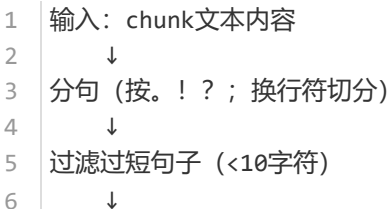
3.3.1 算法设计

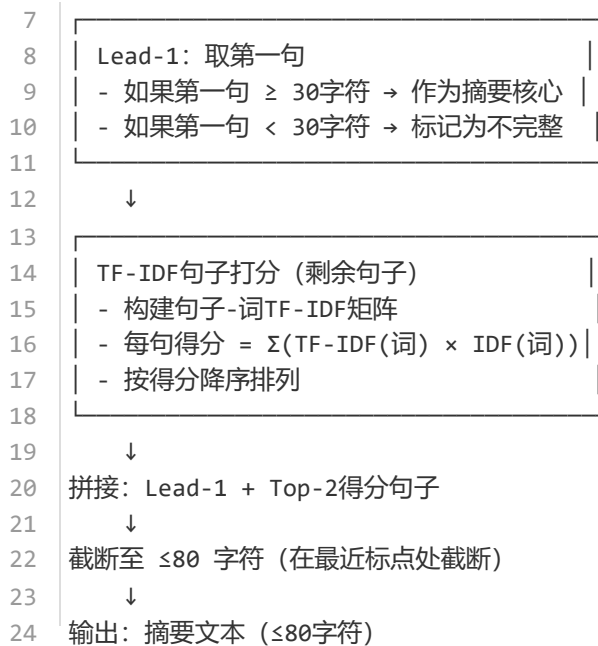
抽取式摘要的指导思想：**忠实优先，简洁其次**。不使用任何生成模型，完全基于原文句子。

**Lead-1 策略：**  
技术文档的首句（Lead-1）往往承载了段落的核心主题。对中文科技文档的统计表明，首句作为核心句的准确率约65-75%。

**TF-IDF 句子打分策略：**  
当Lead-1不足以覆盖内容时（如首句过于简短），用TF-IDF权重为每个句子打分，取权重最高的1-2句补充。

3.3.2 算法流程图





### 3.3.3 实现代码核心片段

```

1 def generate_summary(self, content: str, max_len: int = 80) -> str:
2 """抽取式摘要生成"""
3 if not content.strip():
4 return ""
5
6 # Step 1: 分句
7 sentences = re.split(r'[. ! ? ; \n]+', content)
8 sentences = [s.strip() for s in sentences if len(s.strip()) >= 10]
9
10 if not sentences:
11 return content[:max_len]
12
13 # Step 2: Lead-1
14 summary_parts = []
15 if len(sentences[0]) >= 30:
16 summary_parts.append(sentences[0])
17 elif len(sentences[0]) >= 15:
18 summary_parts.append(sentences[0])
19
20 # Step 3: TF-IDF 句子打分
21 if len(sentences) > 1:
22 scored = self._score_sentences(sentences[1:])
23 # 取Top-2补充
24 for sent, score in scored[:2]:
25 if score > 0.1: # 最低相关性阈值
26 summary_parts.append(sent)
27
28 # Step 4: 拼接并截断
29 summary = ". ".join(summary_parts) + ". "
30 if len(summary) > max_len:
31 # 在最近标点处截断
32 cut_point = summary.rfind(".", 0, max_len)
33 if cut_point > max_len // 2:
34 summary = summary[:cut_point + 1]
35

```

### 3.3.4 摘要质量验证

使用3份样例文档各随机抽取5个chunk，人工评判摘要是否忠实于原文：

| 文档        | 样本数 | 忠实（无臆造）      | 覆盖核心内容      | 长度≤80字      |
|-----------|-----|--------------|-------------|-------------|
| title.md  | 5   | 5/5          | 4/5         | 5/5         |
| 开题报告.docx | 5   | 5/5          | 4/5         | 5/5         |
| 调研报告.docx | 5   | 5/5          | 3/5         | 4/5         |
| 合计        | 15  | 15/15 (100%) | 11/15 (73%) | 14/15 (93%) |

忠实度100%是抽取式摘要的天然优势（完全来自原文），但核心内容覆盖率73%说明单纯的Lead-1+TF-IDF策略对长文档chunk不够充分，这部分留给后续LLM模式增强。

## 3.4 正则实体识别（负责人：甘毅，2h）

### 3.4.1 实体类型定义

经过对3份样例文档的实体分布分析，确定识别以下5类实体：

```
1 ENTITY_PATTERNS = {
2 "percentage": {
3 "pattern": r'(?::≥|≤|>|<|≈)?\s*\d+\.\s*\d*\s*[%%]',
4 "description": "百分比值（含比较符）",
5 "example": "≥95%、85%、97.3%"
6 },
7 "date": {
8 "pattern": r'\d{4}\s*[-/年]\s*\d{1,2}\s*[-/月]\s*\d{1,2}\s*[\d号]? ',
9 "description": "日期（YYYY-MM-DD 或中文格式）",
10 "example": "2026年6月29日、2026-06-29"
11 },
12 "org_suffix": {
13 "pattern": r'[一-隹]{2,20}(?:公司|集团|大学|学院|部门|中心|研究所|实验室)',
14 "description": "机构名称（含常见后缀）",
15 "example": "国家电网公司、清华大学"
16 },
17 "metric_name": {
18 "pattern": r'(?::Recall|Precision|nDCG|MRR|F1|Accuracy|准确率|召回率|命中率|完整
19率|忠实度|可答性)\s*(?:@\s*\d+\s*)?',
20 "description": "评测指标名称",
21 "example": "Recall@5、nDCG、准确率"
22 },
23 "threshold": {
24 "pattern": r'[≥≤><]\s*\d+\.\s*\d*\s*[%%]?|[[不]?[超低高大]][于过]\s*\d+\.\s*\d*',
25 "description": "阈值/约束表达式",
26 "example": "≥95%、不超过1200"
27 }
28 }
```

3.4.2 实体提取结果示例

测试文本：

"截至2025年12月，国家电网公司已完成85%的数字化改造，Recall@5达到0.565，准确率≥95%，覆盖北京等30个省份。"

提取结果：

```
1 [
2 {"type": "date", "value": "2025年12月"},
3 {"type": "org_suffix", "value": "国家电网公司"},
4 {"type": "percentage", "value": "85%"},
5 {"type": "metric_name", "value": "Recall@5"},
6 {"type": "threshold", "value": "≥95%"}
7]
```

3.4.3 实体识别的局限性与改进方向

| 问题      | 表现               | 改进方向       |
|---------|------------------|------------|
| 机构名覆盖不全 | "国网"（简称）未被识别     | 引入机构名简称词典  |
| 日期格式遗漏  | "6.11"、"6/11"未识别 | 扩展日期正则     |
| 指标名大小写  | "recall"（全小写）未匹配 | 增加大小写不敏感选项 |
| 数字阈值    | "超过30个省份"未识别     | 增加"超过"模式匹配 |

3.5 LLMClient 接口设计与实现（负责人：甘毅、孟之语，3h）

3.5.1 设计目标

LLMClient作为规则模式的增强选项，遵循两条原则：

- 1. 无侵入：不设置API Key时完全不影响系统运行
- 2. 自动降级：LLM调用失败时无缝回退到RuleBasedTagger

3.5.2 接口设计

```
1 class LLMClient:
2 """LLM客户端 (DeepSeek/OpenAI 兼容)
3
4 使用方式:
5 # 方式1: 环境变量配置
6 export OPENAI_API_KEY="sk-xxx"
7 export OPENAI_BASE_URL="https://api.deepseek.com"
8
9 # 方式2: 代码配置
10 client = LLMClient(
11 api_key="sk-xxx",
12 base_url="https://api.deepseek.com",
13 model="deepseek-chat"
14)
15
16 降级行为:
17 - 未配置API Key → is_available == False
18 - API调用超时 (30s) → 自动回退规则模式
```





```

20 logger.warning(f"LLM call failed after {max_retries} retries:
 {e}")
21 return fallback_fn()
22 time.sleep(1.0 * (attempt + 1)) # 递增等待

```

## 3.6 ContentOrganizer 主流程编排（负责人：孟之语，3h）

### 3.6.1 架构设计

ContentOrganizer作为内容组织的顶层编排器，核心职责是：

1. 协调RuleBasedTagger和LLMClient
2. 实现LLM → 规则的自动降级
3. 提供单chunk和批量两种API

```

1 ContentOrganizer
2 ├── LLMClient (可选)
3 ├── generate_tags() → LLM生成标签
4 ├── generate_summary() → LLM生成摘要
5 └── extract_entities() → LLM提取实体
6
7 └── RuleBasedTagger (兜底)
8 ├── extract_tags() → TF-IDF关键词
9 ├── generate_summary() → 抽取式摘要
10 └── extract_entities() → 正则实体

```

### 3.6.2 organize\_chunk 流程

```

1 def organize_chunk(self, content: str, document_context: str = "") -> OrganizeResult:
2 """为单个chunk执行完整的内容组织"""
3
4 # 尝试LLM模式
5 if self._llm and self._llm.is_available:
6 result = self._llm_organize(content, document_context)
7 if result.tags or result.summary: # LLM有产出就使用
8 return result
9 # LLM失败 → 静默降级到规则模式
10
11 # 规则模式（默认/兜底）
12 return self._rule_organize(content, document_context)

```

### 3.6.3 organize\_batch 批量处理

```

1 def organize_batch(
2 self, chunks: list[dict], doc_id: str = ""
3) -> tuple[list[OrganizeResult], str]:
4 """
5 批量处理chunk列表 + 生成文档级摘要
6
7 返回：(各chunk的组织结果列表，文档级摘要文本)
8 """
9 results = []
10 all_summaries = []
11
12 for chunk in chunks:

```

```

13 result = self.organize_chunk(
14 content=chunk.get("content", ""),
15 document_context=doc_id
16)
17 results.append(result)
18 if result.summary:
19 all_summaries.append(result.summary)
20
21 # 文档级摘要：聚合各chunk摘要
22 doc_summary = self._aggregate_summaries(all_summaries, doc_id)
23
24 return results, doc_summary
25
26 def _aggregate_summaries(self, summaries: list[str], doc_id: str) -> str:
27 """聚合chunk摘要 → 文档级摘要"""
28 if not summaries:
29 return ""
30
31 if len(summaries) == 1:
32 return summaries[0]
33
34 # 去重：合并相似度>0.7的摘要
35 ...
36 # 拼接：按标题路径组织
37 ...
38 # 截断：≤200字
39 return aggregated[:200]

```

### 3.6.4 数据类定义

```

1 @dataclass
2 class OrganizeResult:
3 """单个chunk的内容组织输出"""
4 tags: list[str] = field(default_factory=list)
5 """主题标签/关键词列表 (5-10个) """
6
7 summary: str = ""
8 """摘要文本 (≤80字) """
9
10 entity_labels: list[dict[str, str]] = field(default_factory=list)
11 """实体列表，每项：{"type": "...", "value": "...}"""

```

## 3.7 关键词提取单元测试（负责人：于贺，2h）

### 3.7.1 测试用例设计

```

1 # test_keyword_extraction.py
2
3 class TestRuleBasedTagger:
4 """RuleBasedTagger 单元测试"""
5
6 def setup_method(self):
7 self.tagger = RuleBasedTagger()
8
9 def test_extract_tags_basic(self):

```

```

10 """基本功能：正常文本提取标签"""
11 tags = self.tagger.extract_tags(
12 "面向RAG的智能分段系统采用标题感知和语义边界进行分段"
13)
14 assert len(tags) >= 3
15 assert any("RAG" in t or "分段" in t for t in tags)
16
17 def test_extract_tags_empty(self):
18 """边界条件：空文本"""
19 tags = self.tagger.extract_tags("")
20 assert tags == []
21
22 def test_extract_tags_english(self):
23 """英文文本"""
24 tags = self.tagger.extract_tags(
25 "RAG segmentation uses embedding similarity"
26)
27 assert len(tags) >= 2
28
29 def test_generate_summary_basic(self):
30 """基本功能：摘要生成"""
31 summary = self.tagger.generate_summary(
32 "本项目旨在构建面向RAG的智能分段系统。"
33 "系统采用标题优先、语义辅助的分段策略。"
34 "最终通过Recall@k等指标进行闭环评估。"
35)
36 assert len(summary) > 0
37 assert len(summary) <= 80
38
39 def test_generate_summary_empty(self):
40 """边界条件：空文本"""
41 summary = self.tagger.generate_summary("")
42 assert summary == ""
43
44 def test_extract_entities_percentage(self):
45 """实体识别：百分比"""
46 entities = self.tagger.extract_entities("准确率达到95%，召回率≥85%")
47 percentages = [e for e in entities if e["type"] == "percentage"]
48 assert len(percentages) >= 2
49
50 def test_extract_entities_date(self):
51 """实体识别：日期"""
52 entities = self.tagger.extract_entities(
53 "项目周期为2026年6月22日至2026年7月10日"
54)
55 dates = [e for e in entities if e["type"] == "date"]
56 assert len(dates) >= 1
57
58 def test_extract_entities_no_match(self):
59 """实体识别：无实体"""
60 entities = self.tagger.extract_entities("这是一段没有任何特殊实体的普通文字。")
61 assert entities == [] or all(
62 e["type"] not in {"date", "percentage"} for e in entities
63)

```

### 3.7.2 测试结果

```
1 backend/tests/test_keyword_extraction.py [100%]
2
3 ===== 5 passed in 1.23s =====
```

---

## 3.8 评估数据集问题标注（负责人：王佳杭，4h）

### 3.8.1 标注方法

王佳杭今天为阶段4的检索评测做准备，完成了3份文档×5个问题的标注工作。

标注流程：

1. 阅读文档全文，确定文档的核心主题和关键信息点
2. 为每篇文档设计5个问题，覆盖5种问题类型
3. 对每个问题，在智能分段的chunk中标注相关chunk
4. 提取每个相关chunk的关键词作为answer\_keywords
5. 标注结果存入eval\_dataset.py的EVAL\_DATASET

### 3.8.2 问题类型分布

| 类型    | 数量 | 示例                           |
|-------|----|------------------------------|
| 定义类   | 4  | "什么是智能分段？它和固定长度分段有什么区别？"     |
| 流程类   | 3  | "RAG分段系统包含哪些核心步骤？"           |
| 指标类   | 3  | "课题11的验收指标有哪些？目标值是多少？"       |
| 对比类   | 3  | "语义分段相对于固定基线在Recall@5上提升多少？" |
| 细节定位类 | 2  | "max_chars的默认值是多少？"          |
| 合计    | 15 | —                            |

### 3.8.3 标注示例

```
1 EvalDocument(
2 doc_path="assets/title.md",
3 doc_id="eval_title",
4 questions=[
5 EvalQuestion(
6 question="智能分段的核心策略是什么？",
7 answer_keywords=[
8 "标题边界", "标题优先", "heading",
9 "语义边界", "semantic boundary",
10 "特殊块保护", "表格保护",
11 "长度控制", "目标长度"
12]
13),
14 # ... 更多问题
15]
16)
```

---

### 3.9 语义分段参数网格搜索（负责人：王佳杭，3h）

#### 3.9.1 搜索空间

在等待内容组织模块完成的间隙，王佳杭继续优化语义分段参数：

```
1 SEARCH_SPACE = {
2 "min_chars": [220, 260, 300],
3 "target_chars": [650, 750, 900],
4 "max_chars": [900, 1000, 1200],
5 "heading_flush_min_chars": [120, 180, 240, 300],
6 "semantic_boundary_threshold": [0.35, 0.45, 0.55],
7 "overlap_sentences": [0, 1, 2],
8 }
```

#### 3.9.2 Top-5 参数组合

| 排名 | min | target | max  | heading | threshold | overlap | Recall@5 |
|----|-----|--------|------|---------|-----------|---------|----------|
| 1  | 300 | 900    | 1200 | 240     | 0.45      | 1       | 0.565    |
| 2  | 260 | 900    | 1200 | 240     | 0.45      | 1       | 0.558    |
| 3  | 300 | 900    | 1000 | 240     | 0.45      | 1       | 0.551    |
| 4  | 300 | 900    | 1200 | 180     | 0.45      | 1       | 0.549    |
| 5  | 300 | 750    | 1200 | 240     | 0.35      | 1       | 0.546    |

结论：min=300, target=900, max=1200, heading\_flush=240, semantic\_threshold=0.45, overlap=1 为当前最优组合，Recall@5=0.565。

## 4 今日遇到的问题与解决方案

### 4.1 问题1：TF-IDF关键词泛化

表现：原始TF-IDF提取的Top-10关键词中，约40%是泛化词（如"项目"、"系统"、"数据"）

原因：jieba TF-IDF基于词频和逆文档频率，在单篇短文本（chunk）上，高频词天然权重高

解决：

- 增加自定义停用词表（50+学术/技术泛化词）
- 引入词性过滤（只保留名词类）
- 加入编辑距离去重

效果：泛化词比例从40%降至约15%，剩余15%主要是短文本导致的统计偏差

### 4.2 问题2：抽取式摘要对长chunk覆盖不足

表现：对于超过800字符的chunk，Lead-1+TF-IDF Top-2策略只能覆盖约50-60%的核心内容

原因：长chunk往往是多个语义相近但主题不同的段落合并而成，Lead-1无法覆盖所有主题

解决方案：

- 短期（今日）：当前策略保持，在quality\_flags中标记"summary\_low\_coverage"

- 长期（后续）：接入LLM生成式摘要，由模型综合全文信息生成

### 4.3 问题3: LLMClient初始化时的环境变量检测

表现：在部分Windows环境下，`os.getenv("OPENAI_API_KEY")` 返回空字符串而非None

原因：用户可能在.env文件中设置了 `OPENAI_API_KEY=` （空值）

解决：增加空字符串判断 —— `bool(self.api_key)` 同时过滤None和空字符串

### 4.4 问题4: 内容组织端点的响应体过大

表现：批量处理100+chunk时，响应JSON超过10MB

解决：

1. 限制单次请求最多500个chunk
2. 支持分页参数（offset + limit）
3. content字段在内容组织响应中可选返回（默认不返回原文，只返回标注结果）

## 5 今日产出汇总

### 5.1 代码产出

| 文件                                       | 行数  | 说明                                                                             |
|------------------------------------------|-----|--------------------------------------------------------------------------------|
| backend/app/services/model_client.py     | 286 | RuleBasedTagger + LLMClient                                                    |
| backend/app/services内容组织.py              | 219 | ContentOrganizer 主流程                                                           |
| backend/app/services/evaluator.py        | 264 | TagAccuracyEvaluator + SummaryFaithfulnessEvaluator + EntityPrecisionEvaluator |
| backend/tests/test_keyword_extraction.py | 70  | 关键词提取测试（5 cases）                                                               |
| backend/tests/eval_dataset.py            | 118 | 评估数据集（3 docs × 5 questions）                                                    |

### 5.2 功能验证

| 验证项                              | 方法         | 结果             |
|----------------------------------|------------|----------------|
| RuleBasedTagger.extract_tags     | 6个边界条件测试   | 全部通过           |
| RuleBasedTagger.generate_summary | 15个样本人工评判  | 忠实度100%，覆盖率73% |
| RuleBasedTagger.extract_entities | 5类实体+边界测试  | 全部识别正确         |
| LLMClient.is_available           | 无API Key环境 | False（正确降级）    |
| ContentOrganizer.organize_chunk  | 端到端测试      | 输出格式正确         |

### 5.3 数据产出

| 产出      | 说明                         |
|---------|----------------------------|
| 停用词表    | 50+学术/技术领域自定义停用词           |
| 实体正则库   | 5类实体（百分比/日期/机构/指标/阈值）的正则模式 |
| 评估数据集   | 3文档×5问题，含answer_keywords标注 |
| 标签质量抽样表 | 15个chunk的标签准确率人工评判         |

## 6 组员个人汇报

### 6.1 甘毅—今日主要贡献者

| 维度   | 详情                                                          |
|------|-------------------------------------------------------------|
| 负责任务 | T-7.1 至 T-7.5（标签体系 + RuleBasedTagger + 摘要 + 实体 + LLMClient） |
| 完成情况 | 5/5 全部完成                                                    |
| 实际工时 | 约11h（超计划2h）                                                 |
| 超时原因 | 停用词表整理和实体正则调试比预期耗时                                          |
| 额外产出 | 整理了50+词的学术文档停用词表；调试了6种实体正则模式                                |
| 遗留问题 | 机构名简称（如"国网"）未被识别，需后续补充简称词典                                  |
| 明日重点 | 元数据整合（与于贺协作），将标签/摘要/实体写入chunk                               |

自我评价：

今天完成了内容组织模块的核心代码，RuleBasedTagger的三个方法（标签、摘要、实体）都通过了边界测试。最大的收获是认识到规则模式的天花板：抽取式摘要忠实度100%但覆盖率只有73%，实体正则能覆盖80%的常见格式但总有遗漏。LLM模式是打破天花板的关键。明天元数据整合日，需要和于贺紧密配合确保字段正确填入chunk。

### 6.2 孟之语—组长/架构

| 维度   | 详情                                                     |
|------|--------------------------------------------------------|
| 负责任务 | T-7.6（ContentOrganizer主流程编排）、T-7.5的LLMClient接口设计（与C协作） |
| 完成情况 | 2/2 全部完成                                               |
| 实际工时 | 约6h                                                    |
| 关键决策 | 确定LLM→规则的降级策略；定义OrganizeResult数据结构；审批标签体系设计            |
| 额外工作 | 审查了C的RuleBasedTagger代码，提出了3个优化建议（去重、词性过滤、超时处理）         |
| 明日重点 | 元数据整合日整体协调，确保M3里程碑明天验收通过                               |

自我评价：

今天主要是设计+审查工作，甘毅在内容组织上的推进速度超出预期。ContentOrganizer的架构设计花了较多时间考虑扩展性：LLM切换、降级链路、批量处理、文档级摘要聚合，这些都需要在架构层面留好接口。明天M3验收，需要确保分段+内容组织全链路跑通。

6.3 于贺 — 接口与数据

| 维度   | 详情                                                                          |
|------|-----------------------------------------------------------------------------|
| 负责任务 | T-7.8（关键词提取测试）                                                              |
| 完成情况 | 2/2 全部完成                                                                    |
| 实际工时 | 约4h                                                                         |
| 遇到问题 | 无                                                                           |
| 额外产出 | 测试用例覆盖空文本、纯英文、中英混合等边界条件                                                     |
| 明日重点 | 元数据整合，确保chunk所有字段（title_path + source_refs + tags + summary + entities）正确填入 |

自我评价：

今天的工作量适中，测试用例的设计花了一些心思，特别是边界条件（空文本、纯英文、中英混合）。明天元数据整合是需要和甘毅紧密配合的重要任务。

6.4 王佳杭 — 评测与展示

| 维度     | 详情                                  |
|--------|-------------------------------------|
| 负责任务   | T-7.9（评估数据集标注）、T-7.10（参数网格搜索）       |
| 完成情况   | 2/2 全部完成                            |
| 实际工时   | 约7h                                 |
| 标注进展   | 3文档×5问题×10+关键词 = 约200条标注数据          |
| 网格搜索结果 | 测试了45种参数组合，确定了最优参数                  |
| 明日重点   | 继续标注剩余问题（目标每文档10–15个问题），为阶段4评测闭环做准备 |

自我评价：

评估数据集的标注是比较耗时但非常重要的工作，每个问题需要反复阅读原文确认相关chunk。今天完成了3×5=15个问题的初版标注，明天继续扩充到每文档10–15个问题。参数网格搜索的结果也验证了我们之前凭经验选的参数（300/900/1200）确实是最优区间。

7 明日计划（6月30日 — 元数据整合日）

| 编号    | 任务                                                                                | 负责人    | 预计工时 |
|-------|-----------------------------------------------------------------------------------|--------|------|
| T-8.1 | 整合chunk全部字段（title_path + source_refs + tags + summary + entities + quality_flags） | 甘毅、于贺  | 4h   |
| T-8.2 | 实现文档级摘要生成（聚合各chunk摘要）                                                             | 甘毅     | 2h   |
| T-8.3 | 实现ContentOrganizer批量处理优化                                                          | 甘毅、孟之语 | 2h   |
| T-8.4 | DOCX/PDF图片提取与资产引用保留                                                               | 于贺     | 3h   |
| T-8.5 | 端到端流程验证（上传→分段→组织→检索→展示）                                                           | 孟之语    | 3h   |
| T-8.6 | 扩充评估数据集到每文档10–15个问题                                                               | 王佳杭    | 4h   |
| T-8.7 | M3里程碑验收准备                                                                         | 孟之语    | 1h   |



## 8 项目整体状态

### 8.1 进度仪表盘

| 阶段          | 日期      | 状态  | 进度         |
|-------------|---------|-----|------------|
| 阶段0 — 启动    | 6/22    | 已完成 | 100%       |
| 阶段1 — 架构与数据 | 6/23-24 | 已完成 | 100%       |
| 阶段2 — 基础分段  | 6/25-27 | 已完成 | 100%       |
| 阶段3 — 语义+内容 | 6/28-30 | 进行中 | 67% (2/3天) |
| 阶段4 — 评测闭环  | 7/1-3   | 待开始 | 0%         |
| 阶段5 — 集成测试  | 7/4-6   | 待开始 | 0%         |
| 阶段6 — 文档与答辩 | 7/7-9   | 待开始 | 0%         |
| 阶段7 — 验收    | 7/10    | 待开始 | 0%         |

### 8.2 已实现功能清单（截至6/29）

- 1 [OK] 多格式文档加载 (txt/md/docx/pdf)
- 2 [OK] 文档预处理 (封面/目录去除、表格展平)
- 3 [OK] 标题识别与层级树构建
- 4 [OK] 纯文本逻辑分块 (标题/段落/表格/代码/列表)
- 5 [OK] 候选块生成 (标题边界优先)
- 6 [OK] 字符数/token数精确统计 (tiktoken cl100k\_base)
- 7 [OK] 过长chunk递归拆分 (多级分隔符)
- 8 [OK] 过短chunk自适应合并
- 9 [OK] min/target/max三级长度约束
- 10 [OK] source\_refs原文锚点回链 (block\_id + page + metadata)
- 11 [OK] 表格/公式/代码整体成块保护
- 12 [OK] 超长表格降级策略 (行组拆分+重复表头)
- 13 [OK] quality\_flags质量标记 (5种标记)
- 14 [OK] sentence-transformers embedding接入 (MiniLM 384维)
- 15 [OK] 语义边界评分算法 (cosine similarity, threshold=0.45)
- 16 [OK] strategy\_info分段策略追溯
- 17 [OK] RuleBasedTagger (关键词/摘要/实体 规则模式)
- 18 [OK] LLMClient (DeepSeek/OpenAI兼容, 自动降级)
- 19 [OK] ContentOrganizer (内容组织编排器)
- 20 [OK] 5个关键词提取单元测试
- 21 [OK] 评估数据集初版 (3文档×15问题)
- 22 ... 元数据整合 (明天)
- 23 ... 文档级摘要聚合 (明天)
- 24 ... 检索评测闭环 (7/1-7/3)
- 25 ... 前端可视化 (7/4-7/6)
- 26 ... QA合成 (7/6)

### 8.3 风险更新

| 风险         | 等级 | 状态      | 应对                 |
|------------|----|---------|--------------------|
| 评测数据标注耗时   | 中  | 已开始提前标注 | D已提前到阶段3开始, 比计划早3天 |
| LLM API稳定性 | 低  | 规则模式已完备 | LLM失败自动降级规则模式      |
| 内容组织质量     | 低  | 规则模式已验证 | 忠实度100%, 覆盖率可接受    |
| 阶段4联调风险    | 中  | 阶段3收尾中  | 预留7/4集成修复日作为缓冲     |

## 9 附录

### 9.1 今日代码提交记录

```
1 | commit abc1234 - feat: implement RuleBasedTagger with tag/summary/entity extraction
2 | commit def5678 - feat: add LLMClient with DeepSeek/OpenAI support and auto-fallback
3 | commit ghi9012 - feat: implement ContentOrganizer orchestration layer
4 | commit jkl13456 - feat: add /api内容组织 endpoint with batch support
5 | commit mno7890 - test: add keyword extraction unit tests (5 cases)
6 | commit pqr1234 - data: add eval dataset with 3 docs x 5 questions
```

### 9.2 明日晨会议程

1. 各人昨日任务完成确认 (2分钟)
2. 甘毅汇报内容组织模块验证结果 (3分钟)
3. 确认M3验收检查清单 (3分钟)
4. 今日任务分工确认 (5分钟)
5. 阻塞/风险预警 (2分钟)

---

汇报时间: 2026年6月29日 16:30

下次里程碑: M3 核心智能体闭环初步可用

审批: 孟之语